

.Server is the object to which the client has direct access

Delegate is the end object that contains the functionality needed by the .client

:There are two types of problems

The server-class doesn't do anything itself and simply creates needless complexity. In this case, give thought to whether this class is needed at .all

Every time a new feature is added to the delegate, you need to create a delegating method for it in the server-class. If a lot of changes are made, .this will be rather tiresome

How to Refactor

Create a getter for accessing the delegate-class object from the server-.class object

Replace calls to delegating methods in the server-class with direct calls .for methods in the delegate-class

Remove Middle Man 🗑️

مشکل (Problem)

یه کلاس تعداد زیادی متد داره که فقط به اشیاء دیگه تفویض (Delegate) می‌کنن.

راه حل (Solution)

این متدها رو حذف کن و کلاینت رو مجبور کن مستقیماً متدهای نهایی رو صدا بزنه.

چرا ریفکتور کنیم؟ (Why Refactor) 🎯

دلیل

توضیح

Server بی خاصیت	کلاس Server هیچ کاری انجام نمیده و فقط پیچیدگی بی مورد ایجاد می‌کنه. اصلاً این کلاس لازمه؟
تغییرات خسته کننده	هر بار ویژگی جدید به Delegate اضافه بشه، باید توی Server هم یه متد تفویضی براش بسازی

How to Refactor (چطور انجام بدیم)

قدم

توضیح

۱	یه getter برای دسترسی به شیء Delegate از Server بساز
۲	صدا زدن‌های متدهای تفویضی در Server رو با صدا زدن مستقیم متدهای Delegate جایگزین کن

مثال

قبل:

csharp

```

public class Department
{
    ;()private Person manager = new

!public string GetManagerName() => manager.GetName(); // Middle Man
{

Client //
;()var dept = new Department
;()var name = dept.GetManagerName

```

بعد:

csharp

```

        public class Department
        {

            مستقیم public Person Manager { get; } = new(); // getter

        }

        Client //
        ;()var dept = new Department
        Delegate به مستقیم به // ;()var name = dept.Manager.GetName
    
```

تمرین

این کد رو با **Remove Middle Man** اصلاح کن:

csharp

```

        public class OrderService
        {

            ;()private PaymentGateway gateway = new

            Middle Man - فقط تفویض می‌کنه
            ;public bool ProcessPayment(decimal amount) => gateway.Pay(amount)
            ;public bool RefundPayment(decimal amount) => gateway.Refund(amount)
            ;public string GetPaymentStatus() => gateway.Status

        }

        public class PaymentGateway
        {

            public bool Pay(decimal amount) { Console.WriteLine($"Paying {amount}"); return true; }

            public bool Refund(decimal amount) { Console.WriteLine($"Refunding {amount}"); return true; }

            ;"public string Status => "Active

        }

        :Client //
        ;()var service = new OrderService
        ;service.ProcessPayment(1000)
    
```

راهنمایی: PaymentGateway رو با getter در دسترس بذار و متدهای تفویضی رو حذف کن.

بفرست تا بررسی کنیم! 📩

خودت حلش کن

کد اصلاح شده با Remove Middle Man

csharp

```
public class OrderService
{
    public PaymentGateway Gateway { get; } = new(); // getter مستقیم
}

public class PaymentGateway
{
    public bool Pay(decimal amount) { Console.WriteLine($"Paying {amount}"); return true; }
    public bool Refund(decimal amount) { Console.WriteLine($"Refunding {amount}"); return true; }
    public string Status => "Active"
}

// Client
var service = new OrderService();
// مستقیماً به Delegate ;service.Gateway.Pay(1000)
```

مقایسه

نسخه اصلاح	کد اصلی	
 getter ۱	۳ متد تفویضی	تعداد متدهای OrderService
 خودکار در دسترسه	باید توی OrderService هم تفویض بسازی	اضافه کردن متد جدید به Gateway

نسخه اصلاح	کد اصلی
✓ کمتر	بیشتر

Remove Middle Man تموم شد! 🎉

و با این، Middle Man بسته شد. و Couplers کامل شد!

وضعیت	بو
✓	Feature Envy
✓	Inappropriate Intimacy
✓	Message Chains
✓	Middle Man

تبریک! تمام Code Smells تموم شد! 🏆

وضعیت	دسته
✓	Bloaters
✓	Object-Orientation Abusers
✓	Change Preventers
✓	Dispensables
✓	Couplers

همه ۵ دسته Code Smell و کلی تکنیک ریفتورینگ رو کار کردیم! 🎉🔧